Novopayment - [BNP] [INTERNO] Análisis de reutilización de SendPassCode para 3DS

NovoPayment, Inc.

Version v1.0.0, 17.03.2026: [BNP] [INTERNO] Análisis de reutilización de SendPassCode para 3DS Documentation

Table of Contents

1. Objetivo . . . 1
2. Contexto del analisis interno . . . 2
   2.1. Contexto . . . 2
   2.2. Alcance del analisis . . . 2
   2.3. Criterios de aceptacion de la historia . . . 2
3. Resumen ejecutivo . . . 3
4. Referencia de flujo . . . 4
   4.1. Convencion visual del comparativo . . . 4
5. Hallazgos del comportamiento actual . . . 5
   5.1. Funcionamiento end-to-end (E2E) . . . 5
6. Plataformas de envio (email/sms) . . . 7
7. Dependencias y restricciones fuera de tokenization . . . 8
8. Analisis contra la necesidad de 3DS . . . 9
9. Evidencia tecnica del endpoint actual . . . 10
   9.1. Endpoint . . . 10
   9.2. Path params . . . 10
   9.3. Request body . . . 10
   9.4. Alineacion con referencia externa (SendPasscode) . . . 11
      9.4.1. Ejemplo de request . . . 11
   9.5. Contenido esperado dentro de `encryptedData` (desencriptado) . . . 12
   9.6. Responses . . . 13
      9.6.1. 200 OK (flujo normal) . . . 13
      9.6.2. 200 OK (body invalido tras decrypt) . . . 13
      9.6.3. Error de negocio (`NovoException`) . . . 13
      9.6.4. 500 Internal Server Error . . . 13
10. Evaluacion de factibilidad para 3DS . . . 15
   10.1. Pros de una reutilizacion parcial . . . 15
   10.2. Contras y limitaciones . . . 15
   10.3. Riesgos de reutilizarlo directamente en 3DS . . . 16
   10.4. cURL de ejemplo . . . 16
   10.5. cURLs de referencia observados . . . 16
      10.5.1. SendPassCode (proceso actual segun referencia externa) . . . 16
      10.5.2. Notification (flujo actual relacionado) . . . 18
      10.5.3. SendPassCode (flujo actual observado) . . . 19

11. Factibilidad de reutilizacion para 3DS . . . 21
11.1. Por que no es reutilizable de forma directa . . . 21
11.2. Que si se puede reutilizar . . . 21
11.3. Recomendacion . . . 21
11.4. Clasificacion final de la recomendacion . . . 22
11.5. Validacion contra criterios de aceptacion . . . 22
12. Anexo: flujo 3DS propuesto (E2E) . . . 23
12.1. Flujo funcional propuesto . . . 23
12.2. Reglas de decision minimas (3DS) . . . 23
12.3. Trazabilidad propuesta para 3DS . . . 24
12.4. Respuestas esperadas del endpoint 3DS . . . 24
13. Plan B: adaptacion del endpoint actual si no aprueban endpoint 3DS dedicado . . . 25
13.1. Estrategia de adaptacion recomendada . . . 25
13.2. Riesgos residuales del Plan B . . . 25
13.3. Mitigaciones minimas para aplicar Plan B . . . 26
14. Estimacion de cambios de codigo para implementar 3DS sobre el flujo actual . . . 27
14.1. Alcance considerado para la estimacion . . . 27
14.2. Impacto por componente . . . 27
14.3. Totales estimados . . . 28
14.4. Escenarios de esfuerzo . . . 28

Chapter 1. Objetivo

Documentar, con base en la historia interna y en la implementacion actual, si el endpoint `POST /{tenant}/{customerType}/tokenization/sendPassCode` puede reutilizarse en el servicio de 3DS o si requiere un endpoint/flujo dedicado.

Chapter 2. Contexto del analisis interno

Item	Detalle
Historia	`[BNP] [INTERNO] Analisis de reutilizacion de SendPassCode para 3DS`
Ticket origen	`CEB-4255: [BNP] / pa-bnp / 3DS / SendOTP: Incluir en el proceso de envio de Mensajes Email & SMS del Banco, las peticiones de SendOTP`
Objetivo del analisis	Determinar si `SendPassCode` puede reutilizarse en el servicio de 3DS o si se requiere un endpoint/flujo especifico.
Referencia funcional externa	`https://developer.novopayment.com/api/tokenization/issuer-tokenization-api-v1.0#/operations/SendPasscode`

2.1. Contexto

Se solicito evaluar si el endpoint `SendPassCode`, hoy implementado en el contexto de tokenizacion, puede cubrir la necesidad de envio de codigo/validacion requerida por 3DS.

2.2. Alcance del analisis

Este documento cubre:

- Objetivo funcional actual del endpoint y flujo donde se usa hoy.
- Dependencias tecnicas y de negocio necesarias para operarlo.
- Validacion de capacidades relevantes para 3DS: canal de envio, expiracion, control de intentos y trazabilidad.
- Impactos, riesgos y limitaciones de reutilizar el flujo actual en 3DS.
- Recomendacion tecnica final: reutilizable sin cambios, reutilizable con ajustes o no reutilizable.

2.3. Criterios de aceptacion de la historia

- Se documenta el comportamiento actual de `SendPassCode` y su alcance real.
- Se documentan las dependencias y restricciones para usarlo fuera de tokenization.
- Se entrega una recomendacion tecnica clara sobre su reutilizacion en 3DS, con pros, contras y riesgos.

Chapter 3. Resumen ejecutivo

Punto	Conclusion
Decision principal	`SendPassCode` **no debe reutilizarse directamente** como endpoint de negocio para 3DS.
Reutilizacion viable	**Parcial**, solo a nivel de componentes internos (criptografia, seleccion de canal e integracion con `Message Connector`).
Motivo principal	El flujo actual esta acoplado al dominio de tokenizacion en contrato, categoria, lookup y persistencia.
Recomendacion	Crear un endpoint/flujo especifico para 3DS y reutilizar internamente solo componentes comunes.

Chapter 4. Referencia de flujo

Este endpoint tiene dos vistas complementarias de flujo:

- Secuencia end-to-end: `docs/diagram/sequence/sendPassCode-tokenization-e2e.puml`
- Diagrama de actividad (flujo puro): `docs/diagram/sequence/sendPassCode-tokenization-activity.puml`
- Diagrama comparativo (actual vs propuesta 3DS): `docs/diagram/sequence/sendPassCode-tokenization-vs-3ds.puml`

4.1. Convencion visual del comparativo

- Azul: flujo actual de `sendPassCode` en tokenization.
- Naranja: flujo propuesto para reutilizacion controlada en 3DS.

Chapter 5. Hallazgos del comportamiento actual

5.1. Funcionamiento end-to-end (E2E)

El flujo funcional de `POST /{tenant}/{customerType}/tokenization/sendPassCode` es:

1. El controlador recibe `SendPassCodeTokenization`, registra logs y carga configuracion por tenant con `configService(tenantId)`.
2. Se valida y procesa `encryptedData` con `verify(…)` y luego `decrypt(…)`.
3. El JSON desencriptado se mapea a `EncryptedDataSendPass`.
   a. Si el mapeo falla, responde `HTTP 200` sin body.
4. Se calcula el canal de envio (`msgType`) desde `otpDestiny`:
   a. Si `isEmail(otpDestiny)=true`, canal `email`.
   b. En caso contrario, canal `sms`.
5. Se evalua `notificationCategories.emulate`:
   a. Si `emulate=true`, responde `rc=0`, `msg=OK` sin envio real ni persistencia.
   b. Si `emulate=false`, continua flujo real.
6. En flujo real se invoca `setNewSendPassCodeTokenization(…)`.
7. El servicio valida la categoria `tkSendPassCode`:
   a. Si esta inactiva, retorna mensaje `Notification category: tkSendPassCode Disabled`.
   b. Si esta activa, continua.
8. Se consulta cliente por PAN con `getCustomerInfoTokenization(…)` en repositorio.
   a. Si ocurre `NovoException` en esta consulta, se lanza `CUSTOMER_ID_NOT_FOUND`.
9. Se envia OTP segun canal y configuracion activa de categoria:
   a. Email → `sendEmailSendPassCodeTokenization` → provider `novo`.
   b. SMS → `sendSmsSendPasscodeTokenization` → provider `payHubBnp`.
   c. Si el canal correspondiente no esta activo en configuracion, no se envia mensaje.
10. Se guarda traza de tokenizacion en BD con `saveTokenizationNotification(…)`.
    a. Si falla el guardado de traza, se registra en log y el flujo principal continua.
11. Se responde `HTTP 200` con `NovoResponse` (`rc=0`, `msg=OK`) y `data` con estado de categoria.

Manejo de errores del endpoint:

- `NovoException`: responde con HTTP segun `ResponseType` y body de negocio.
- Excepcion generica: responde `HTTP 500` con mensaje de error.

Chapter 6. Plataformas de envio (email/sms)

La seleccion del canal para `sendPassCode` se define por `otpDestiny`:

- Si `otpDestiny` es email (`isEmail=true`), se usa canal `email`.
- En caso contrario, se usa canal `sms`.

En `sendPassCode` los providers quedan asi:

Flujo	Canal	Metodo en servicio	Provider (`$PROVIDERS$`)
`/tokenization/sendPassCode`	Email	`sendEmailSendPassCodeTokenization`	`novo`
`/tokenization/sendPassCode`	SMS	`sendSmsSendPasscodeTokenization`	`payHubBnp`

Comparativo rapido con otros flujos del mismo servicio:

Flujo	Canal	Metodo en servicio	Provider (`$PROVIDERS$`)
`/tokenization/notifications`	Email	`sendEmailNotificationTokenization`	`novo`
`/tokenization/notifications`	SMS	`sendSmsNotificationTokenization`	`payHubBnp`
Notificaciones generales	Email	`sendEmail`	`""` (sin provider explicito)
Notificaciones generales	SMS	`sendSms`	`twilio`

El provider real final depende del ruteo/configuracion del Message Connector; aqui se documenta el valor que esta seteando este servicio en la URL.

Chapter 7. Dependencias y restricciones fuera de tokenization

El uso actual de `sendPassCode` depende de componentes y supuestos propios del dominio de tokenizacion:

- Configuracion por tenant (`configService`) y categoria fija `tkSendPassCode`.
- Validacion criptografica basada en `verify(…)` y `decrypt(…)` con `tokenizationSettings`.
- Contrato de entrada `SendPassCodeTokenization`.
- Mapeo del payload desencriptado a `EncryptedDataSendPass`.
- Consulta de cliente por PAN mediante `getCustomerInfoTokenization(…)`.
- Persistencia de auditoria mediante `saveTokenizationNotification(…)`.
- Envio por `Message Connector` con providers actuales `novo` y `payHubBnp`.

Estas dependencias implican que el endpoint no es un servicio generico de OTP, sino un flujo especializado de tokenizacion.

Chapter 8. Analisis contra la necesidad de 3DS

La historia solicita validar si el comportamiento actual de `SendPassCode` cubre la necesidad de envio de codigo/validacion para 3DS. El resultado del analisis es el siguiente:

- **Canal de envio**: cubierto parcialmente; el endpoint ya decide entre email y SMS usando `otpDestiny`.
- **Expiracion**: cubierta solo como dato transportado/persistido; no existe logica de vencimiento.
- **Control de intentos**: no cubierto; `otpMaxReached` no acciona reglas funcionales.
- **Trazabilidad**: cubierta parcialmente; existe logging y persistencia, pero con semantica de tokenization.
- **Dominio funcional 3DS**: no cubierto; faltan contrato, estados y trazabilidad propios de autenticacion 3DS.

Por lo anterior, el endpoint actual sirve como **evidencia de capacidades reutilizables**, pero no como implementacion final adecuada para 3DS.

Chapter 9. Evidencia tecnica del endpoint actual

9.1. Endpoint

```
POST /{tenant}/{customerType}/tokenization/sendPassCode
Content-Type: application/json
Accept: application/json
```

9.2. Path params

Parametro	Descripcion
`tenant`	Identificador del banco/tenant (usado para configuracion y llaves criptograficas).
`customerType`	Tipo de cliente requerido por el routing base del controlador.

9.3. Request body

El body corresponde a la entidad `SendPassCodeTokenization`.

Campo	Tipo	Descripcion
`tokenRequestorID`	string	Identificador del token requestor.
`tokenReferenceID`	string	Referencia del token.
`panReferenceID`	string	Referencia del PAN.
`clientWalletAccountID`	string	Cuenta wallet del cliente.
`otpMaxReached`	string	Indicador/estado de maximos intentos OTP.
`otpMethodIdentifier`	string	Canal/metodo OTP.
`otpValue`	string	OTP a notificar.
`otpDestiny`	string	Destino OTP (email o telefono). Si es email, el endpoint enruta por canal email; de lo contrario por sms.
`otpReason`	string	Motivo OTP.
`otpExpirationDate`	string	Fecha de expiracion OTP.

Campo	Tipo	Descripcion
`deviceInfo`	object	Objeto con metadata del dispositivo (se usa al persistir log).
`encryptedData`	string	Payload cifrado/firmado que al desencriptar debe mapear a `EncryptedDataSendPass`.

En la referencia externa de `SendPasscode` puede verse `encryptedData` como objeto JSON en claro. En esta implementacion de `notification-core`, el endpoint procesa `encryptedData` como **string** (JWS/JWE), luego hace `verify` y `decrypt` para mapear a `EncryptedDataSendPass`.

9.4. Alineacion con referencia externa (SendPasscode)

El siguiente cuadro resume diferencias de contrato entre la referencia externa y la implementacion actual evaluada:

Campo	Referencia externa	Implementacion actual (`notification-core`)
`otpMaxReached`	Boolean (`true/false`)	Se maneja como `String` en entidad de entrada/salida interna.
`OtpReason`	Se muestra con `O` mayuscula en algunos ejemplos	Se recomienda enviar `otpReason` (camelCase) para consistencia de mapeo.
`encryptedData`	Objeto JSON en ejemplo	Se espera `String` firmado/cifrado; el objeto va dentro del contenido desencriptado.
`tokenInfo`	Puede verse al mismo nivel del request en ejemplo externo	En esta implementacion, `tokenInfo` debe viajar en el payload interno de `encryptedData`.

Para pruebas de integracion sobre este servicio, usar el contrato operativo local (tabla anterior), aunque la fuente externa se mantenga como referencia funcional del dominio.

9.4.1. Ejemplo de request

```
{
  "tokenRequestorID": "50123456789",
  "tokenReferenceID": "TRX-9ad77f7f",
```

```
  "panReferenceID": "PAN-ff28a4",
  "clientWalletAccountID": "wallet-user-001",
  "otpMaxReached": "false",
  "otpMethodIdentifier": "SMS",
  "otpValue": "184392",
  "otpDestiny": "+573001112233",
  "otpReason": "TOKEN_PROVISIONING",
  "otpExpirationDate": "2026-03-17T21:10:00Z",
  "deviceInfo": {
    "deviceID": "A1B2C3D4E5"
  },
  "encryptedData": "<JWS que contiene JWE>"
}
```

9.5. Contenido esperado dentro de `encryptedData` (desencriptado)

Internamente el controlador hace `verify` + `decrypt`, y el JSON resultante se mapea a `EncryptedDataSendPass`.

```
{
  "cardholderInfo": {
    "primaryAccountNumber": "4111111111111111",
    "name": "JOHN DOE",
    "otpDestiny": null,
    "highValueCustomer": false
  },
  "riskInformation": {
    "accountHolderName": "JOHN DOE",
    "visaTokenScore": "05",
    "visaTokenDecisioning": "APPROVE"
  },
  "tokenInfo": {
    "token": "4761739001010010",
    "tokenType": "NETWORK",
    "tokenStatus": "ACTIVE",
    "tokenAssuranceMethod": "RISK_BASED_AUTH",
    "numberOfActiveTokensForPAN": 1,
    "numberOfInactiveTokensForPAN": 0,
    "numberOfSuspendedTokensForPAN": 0,
    "autoFillIndicator": true
  }
}
```

9.6. Responses

9.6.1. 200 OK (flujo normal)

Cuando no hay excepcion, la respuesta es `NovoResponse` serializada.

```
{
  "rc": "0",
  "msg": "OK",
  "data": "Notification category: tkSendPassCode Activate"
}
```

Notas:

- Si la categoria `tkSendPassCode` esta inactiva, `data` cambia a `Notification category: tkSendPassCode Disabled`.
- Si `notificationCategories.emulate=true`, se devuelve 200 con `rc/msg` y sin ejecutar envio/persistencia (normalmente `data` ausente).

9.6.2. 200 OK (body invalido tras decrypt)

Si falla el mapeo del JSON desencriptado a `EncryptedDataSendPass`, el controlador retorna `HTTP 200` sin body.

9.6.3. Error de negocio (`NovoException`)

Si se lanza `NovoException` (por ejemplo, cliente no encontrado), el endpoint responde con el HTTP code asociado a `ResponseType` y el body de error.

Ejemplo referencial de estructura:

```
{
  "rc": "<codigo_negocio>",
  "msg": "<descripcion>"
}
```

9.6.4. 500 Internal Server Error

Ante excepcion no controlada:

```
<mensaje de la excepcion>
```

Chapter 10. Evaluacion de factibilidad para 3DS

Capacidad requerida por 3DS	Cobertura actual en `sendPassCode`	Observacion
Canal de envio	Parcial	Si soporta email o SMS segun `otpDestiny`, y reutiliza `Message Connector`.
Expiracion OTP	Parcial	Recibe y persiste `otpExpirationDate`, pero no aplica validacion ni gestion funcional de vencimiento.
Control de intentos	No	Recibe/persiste `otpMaxReached`, pero no implementa conteo, bloqueo, incremento ni reglas de intentos.
Trazabilidad	Parcial	Existe logging, `requestId` y persistencia, pero la traza queda modelada como tokenization, no como autenticacion 3DS.
Contrato 3DS	No	El request/response y el payload desencriptado estan orientados a tokenizacion, no a challenge/autenticacion 3DS.

10.1. Pros de una reutilizacion parcial

- Ya existe integracion para envio OTP por email y SMS.
- Ya existe infraestructura de `verify(…)` / `decrypt(…)`.
- Ya existe lookup de cliente por PAN.
- Ya existe integracion centralizada con `Message Connector`.

10.2. Contras y limitaciones

- El contrato actual esta acoplado al dominio de tokenizacion.
- La categoria funcional es fija (`tkSendPassCode`).
- La persistencia actual registra eventos en estructura de tokenization.
- El endpoint no implementa logica completa de expiracion, intentos ni validacion OTP.
- No contempla atributos y estados propios de 3DS.

10.3. Riesgos de reutilizarlo directamente en 3DS

- Mezcla de dominios de negocio: tokenizacion y autenticacion 3DS.
- Auditoria y trazabilidad ambiguas al persistir eventos 3DS como tokenization.
- Incremento de acoplamiento y riesgo de regresion sobre el flujo actual de tokenization.
- Necesidad de introducir cambios de negocio en un endpoint ya operativo para otro caso de uso.

10.4. cURL de ejemplo

```
curl -X POST "https://<host>/<tenant>/<customerType>/tokenization/sendPassCode" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{
    "tokenRequestorID": "50123456789",
    "tokenReferenceID": "TRX-9ad77f7f",
    "panReferenceID": "PAN-ff28a4",
    "clientWalletAccountID": "wallet-user-001",
    "otpMaxReached": "false",
    "otpMethodIdentifier": "SMS",
    "otpValue": "184392",
    "otpDestiny": "+573001112233",
    "otpReason": "TOKEN_PROVISIONING",
    "otpExpirationDate": "2026-03-17T21:10:00Z",
    "deviceInfo": {"deviceID": "A1B2C3D4E5"},
    "encryptedData": "<JWS que contiene JWE>"
  }'
```

10.5. cURLs de referencia observados

10.5.1. SendPassCode (proceso actual segun referencia externa)

```
curl --request POST \
  --url https://urlissuer/v1/sendPasscode \
  --header 'Content-Type: application/json' \
  --data '{
  "tokenRequestorID": 40022358888,
  "tokenReferenceID": "DNITHE30194726875288888",
  "panReferenceID": "V-3019231458906209630635",
  "clientWalletAccountID": "string",
  "otpMaxReached": true,
  "otpMethodIdentifier": "string",
```

```
  "otpValue": "string",
  "OtpReason": "PROVISIONING",
  "otpExpirationDate": "string",
  "deviceInfo": {
    "deviceID": "CTF0000000001",
    "deviceIndex": 99,
    "deviceLanguageCode": "eng",
    "deviceType": "PHONE",
    "deviceName": "Novo_Phone",
    "deviceNumber": 6506198963,
    "osType": "ANDROID",
    "osVersion": "4.4.4",
    "osBuildID": 13.1,
    "deviceIDType": "TEE",
    "deviceManufacturer": "Samsung",
    "deviceBrand": "Galaxy",
    "deviceModel": "M05",
    "deviceLocation": "string",
    "deviceIPAddressV4": "127.0.0.1",
    "locationSource": "WIFI",
    "tokenProtectionMethod": "CLOUD",
    "originalDeviceID": "...",
    "originalDeviceType": "..."
  },
  "encryptedData": {
    "cardholderInfo": {
      "primaryAccountNumber": "0000000000000000",
      "otpDestiny": "57300800800"
    }
  },
  "tokenInfo": {
    "token": "...",
    "tokenType": "ECOMMERCE",
    "tokenStatus": "ACTIVE",
    "tokenExpirationDate": {
      "month": 7,
      "year": 2016
    },
    "tokenRequestorName": "NovoPayment Wallet",
    "tokenAssuranceMethod": "00",
    "numberOfActiveTokensForPAN": 2,
    "numberOfInactiveTokensForPAN": 2,
    "numberOfSuspendedTokensForPAN": 0,
    "tokenActivationDate": "2022-04-25T23:19:12.000Z",
    "tokenDeactivationDate": "...",
    "lastTokenStatusUpdatedTime": "...",
```

```
    "originalToken": "...",
    "originalTokenRequestorID": "...",
    "originalTokenReferenceID": "...",
    "originalTokenType": "SECURE_ELEMENT",
    "originaltokenAssuranceMethod": "00",
    "autoFillIndicator": false
  }
}'
```

El cURL anterior corresponde al proceso actual de `SendPassCode` documentado en la referencia externa. Para ejecutar contra `notification-core`, ver el cURL operativo local y la seccion de alineacion de contrato.

10.5.2. Notification (flujo actual relacionado)

```
curl --location 'https://banco-nacional-pa.novopayment.net/notification-
core/v1/bnp_pa/retail/tokenization/notifications' \
--header 'Content-Type: application/json' \
--header 'x-context-id: bnp_pa' \
--data '{
  "eventType": "TOKEN_CREATED",
  "panReferenceID": "V-3021084701796922123456",
  "tokenReferenceID": "DNITHE3016161784390573110200000",
  "tokenRequestorID": 40000000055,
  "messageReason": "Token created successfully",
  "messageReasonCode": "TOKEN_CREATED",
  "dateTimeOfEvent": "2022-04-25T23:19:12.000Z",
  "deviceInfo": {
    "deviceID": "OTc0MzJhMmEtYWUxOC0zMGUy",
    "deviceLanguageCode": "eng",
    "deviceType": "MOBILEPHONE_OR_TABLET",
    "deviceName": "TTIxMDJKMjBTRw..",
    "osType": "ANDROID",
    "osVersion": 11,
    "osBuildID": "RKQ1.200826.002",
    "deviceIDType": "Derived",
    "deviceManufacturer": "Xiaomi",
    "deviceBrand": "POCO",
    "deviceModel": "M2102J20SG",
    "tokenProtectionMethod": "SOFTWARE",
    "actionCode": 0,
    "lifeCycleTraceID": 302291744095994,
    "clientWalletAccountID": "7f7e9cef-1234",
    "panSource": "KEY_ENTERED",
```

```
    "cvv2ResultsCode": "M",
    "consumerEntryMode": "UNKNOWN",
    "locale": "en_US",
    "termsAndConditions": {
      "id": "3f7f87ae2ff0439f80f322ec3dfa8777",
      "date": "1970-01-03T10:45:18.000+0000"
    },
    "tokenRequestorTspID": 40000000055,
    "walletAccountEmailAddressHash":
"A9C49C43A63249115F6DD9CD72C6A8AC58F5D44E35EDC6D36CFF999BAC3B228E",
    "addressVerificationResultCode": "A"
  },
  "encryptedData": "<payload omitido>"
}'
```

10.5.3. SendPassCode (flujo actual observado)

```
curl --location 'https://banco-nacional-pa.novopayment.net/notification-
core/v1/bnp_pa/retail/tokenization/sendPassCode' \
--header 'Content-Type: application/json' \
--header 'x-context-id: bnp_pa' \
--data-raw '{
  "tokenRequestorID": 40022358888,
  "tokenReferenceID": "DNITHE30194726875288888",
  "panReferenceID": "V-3019231458906209630635",
  "clientWalletAccountID": "string",
  "otpMaxReached": true,
  "otpMethodIdentifier": "string",
  "otpValue": "12345",
  "otpDestiny": "joaquinteromo552@gmail.com",
  "OtpReason": "PROVISIONING",
  "otpExpirationDate": "string",
  "deviceInfo": {
    "deviceID": "CTF0000000001",
    "deviceIndex": 99,
    "deviceLanguageCode": "eng",
    "deviceType": "PHONE",
    "deviceName": "Novo_Phone",
    "deviceNumber": 6506198963,
    "osType": "ANDROID",
    "osVersion": "4.4.4",
    "osBuildID": 13.1,
    "deviceIDType": "TEE",
    "deviceManufacturer": "Samsung",
```

```
    "deviceBrand": "Galaxy",
    "deviceModel": "M05",
    "deviceLocation": "string",
    "deviceIPAddressV4": "127.0.0.1",
    "locationSource": "WIFI",
    "tokenProtectionMethod": "CLOUD",
    "originalDeviceID": "...",
    "originalDeviceType": "..."
  },
  "encryptedData": "<payload omitido>"
}'
```

Los ejemplos anteriores son payloads de referencia observados y se documentan para contrastar el uso actual del endpoint con la necesidad de 3DS.

Chapter 11. Factibilidad de reutilizacion para 3DS

Conclusión: **no es recomendable reutilizar este endpoint de forma directa para implementar 3DS**; **sí es factible reutilizar partes internas del flujo**.

11.1. Por que no es reutilizable de forma directa

Este endpoint esta acoplado al caso de uso de tokenization/sendPassCode por varias razones:

- El contrato de entrada esta definido por `SendPassCodeTokenization`, con campos y semantica orientados a tokenizacion.
- El `encryptedData` se valida y se mapea a `EncryptedDataSendPass`, cuyo contenido esperado tambien responde al dominio de tokenizacion.
- La categoria funcional usada por el servicio es fija: `tkSendPassCode`.
- La persistencia se realiza con `saveTokenizationNotification(…)`, que guarda una traza con estructura de tokenization y no una de autenticacion 3DS.
- La logica actual determina canal solo a partir de `otpDestiny` (email vs sms), pero no contempla atributos propios de 3DS como `challenge`, `transaction status`, `acs`, `ds`, `cavv`, `eci`, `transStatusReason` o correlacion especifica de autenticacion.

11.2. Que si se puede reutilizar

Hay componentes que podrian reaprovecharse para una implementacion 3DS:

- Verificacion y desencriptado (`verify` / `decrypt`) si 3DS usa el mismo esquema JWS/JWE y administracion de llaves.
- Resolucion del canal de envio a partir del destino (`isEmail`).
- Integracion con `Message Connector` para envio por email o SMS.
- Consulta de cliente por PAN, siempre que el caso de uso 3DS mantenga esa forma de identificacion.

11.3. Recomendacion

Para 3DS, lo aconsejable es crear un **endpoint y contrato propios**, por ejemplo una categoria dedicada y un modelo de request/response alineado al dominio 3DS. Internamente se pueden reutilizar piezas comunes del flujo actual (criptografia, seleccion de canal y adaptacion al `Message Connector`), pero evitando reutilizar sin cambios el endpoint `sendPassCode`, porque mezclaria dos

dominios de negocio distintos: **tokenizacion** y **autenticacion 3DS**.

11.4. Clasificacion final de la recomendacion

Opcion	Resultado
Reutilizable sin cambios	No
Reutilizable con ajustes	Solo a nivel de componentes internos, no como endpoint de negocio
No reutilizable / requiere endpoint dedicado	Si

11.5. Validacion contra criterios de aceptacion

- **Criterio 1 cumplido**: se documento el comportamiento actual de `SendPassCode` y su alcance real.
- **Criterio 2 cumplido**: se documentaron dependencias y restricciones para usarlo fuera de tokenization.
- **Criterio 3 cumplido**: se entrega una recomendacion tecnica clara con pros, contras, limitaciones y riesgos.

Chapter 12. Anexo: flujo 3DS propuesto (E2E)

Como complemento de la recomendacion, se propone un flujo 3DS dedicado (ejemplo referencial: `POST /{tenant}/{customerType}/3ds/sendOtp`) para separar dominio de autenticacion 3DS del dominio de tokenization.

12.1. Flujo funcional propuesto

1. Recibir request 3DS con contrato propio (datos de autenticacion y de notificacion).
2. Validar estructura minima obligatoria (`threeDSServerTransID`, `transactionId`, destino OTP, metadata de canal).
3. Validar firma/cifrado (`verify` / `decrypt`) reutilizando el componente criptografico actual, si aplica al canal 3DS.
4. Resolver canal de envio (`email` o `sms`) segun regla de negocio 3DS y destino recibido.
5. Consultar configuracion de categoria 3DS (nueva categoria, separada de `tkSendPassCode`).
6. Consultar estado de la transaccion 3DS y contador de intentos.
7. Evaluar reglas de control:
   a. Si OTP expirado → rechazar con estado de expiracion.
   b. Si intentos excedidos → rechazar con estado de bloqueo.
   c. Si reglas validas → continuar envio.
8. Enviar OTP por `Message Connector` (email/sms) reutilizando adaptadores de canal existentes.
9. Persistir trazabilidad 3DS en estructura dedicada (no `saveTokenizationNotification`).
10. Responder con contrato 3DS (estado de operacion y codigos de negocio de autenticacion).

12.2. Reglas de decision minimas (3DS)

Condicion	Accion	Resultado esperado
Canal no permitido por politica 3DS	No enviar OTP	Respuesta de rechazo funcional
OTP expirado	No enviar OTP	Respuesta de expiracion
Intentos maximos alcanzados	No enviar OTP	Respuesta de bloqueo por intentos

Condicion	Accion	Resultado esperado
Canal valido + OTP vigente + intentos disponibles	Enviar OTP	Respuesta de exito

12.3. Trazabilidad propuesta para 3DS

Se recomienda persistir, como minimo:

- `requestId` (cabecera de correlacion).
- `threeDSServerTransID` (correlacion 3DS extremo a extremo).
- `transactionId`/`dsTransID` (segun contrato final).
- Canal seleccionado, proveedor usado y resultado de entrega.
- Estado de intento (numero de intento, disponible/restante, vencimiento OTP).
- Estado final de autenticacion relacionado al envio OTP.

12.4. Respuestas esperadas del endpoint 3DS

Se recomienda definir un contrato explicito con estados de negocio 3DS. Como base minima:

- Exito de envio OTP.
- Rechazo por expiracion de OTP.
- Rechazo por intentos maximos.
- Rechazo por canal no habilitado.
- Error de negocio (datos o estado de transaccion invalido).
- Error tecnico (falla de integracion).

Este anexo mantiene la reutilizacion tecnica parcial (criptografia, canal y conector), pero separa contrato, reglas y trazabilidad del dominio de tokenization.

Chapter 13. Plan B: adaptacion del endpoint actual si no aprueban endpoint 3DS dedicado

Si la propuesta de endpoint dedicado es rechazada, se recomienda una adaptacion controlada sobre `sendPassCode` para reducir riesgo de regresion y mantener compatibilidad con el flujo actual.

13.1. Estrategia de adaptacion recomendada

1. Introducir un discriminador de caso de uso en request (ejemplo: `useCase=TOKENIZATION|THREEDS`).
   a. Si no llega `useCase`, asumir `TOKENIZATION` por compatibilidad hacia atras.
2. Separar la orquestacion de negocio por flujo interno:
   a. `TOKENIZATION` → flujo actual sin cambios funcionales.
   b. `THREEDS` → flujo nuevo con reglas 3DS.
3. Extender contrato para 3DS sin romper clientes actuales:
   a. Identificadores de correlacion 3DS (por ejemplo `threeDSServerTransID`, `transactionId` /`dsTransID`).
   b. Parametros funcionales de OTP (TTL/expiracion real e intentos maximos).
4. Implementar reglas de negocio 3DS que hoy no existen en el endpoint:
   a. Validacion de expiracion OTP antes de envio.
   b. Control de intentos (conteo, incremento y bloqueo por maximos).
   c. Idempotencia por correlacion transaccional para reintentos tecnicos.
5. Mantener trazabilidad separada para 3DS:
   a. Crear persistencia dedicada 3DS (evitar registrar 3DS solo en `saveTokenizationNotification(…)`).
6. Ajustar respuesta de negocio para flujo 3DS:
   a. Incluir estado funcional (ejemplo: `OTP_SENT`, `OTP_EXPIRED`, `ATTEMPTS_EXCEEDED`, `CHANNEL_NOT_ALLOWED`).

13.2. Riesgos residuales del Plan B

- Mezcla parcial de dominios dentro del mismo endpoint publico.
- Mayor complejidad de mantenimiento (doble flujo en una misma operacion).

- Mayor superficie de pruebas de regresion en tokenization.

13.3. Mitigaciones minimas para aplicar Plan B

- Feature flag por tenant para activar gradualmente el modo `THREEDS`.
- Suite de pruebas separada por flujo (`TOKENIZATION` y `THREEDS`).
- Monitoreo con metricas diferenciadas por `useCase`.

Chapter 14. Estimacion de cambios de codigo para implementar 3DS sobre el flujo actual

Esta estimacion corresponde al escenario **Plan B** (reutilizar endpoint actual) y se expresa como rango por dependencia de definiciones funcionales finales de 3DS.

14.1. Alcance considerado para la estimacion

- Mantener `POST /tokenization/sendPassCode` como endpoint publico.
- Incorporar discriminador de flujo (`TOKENIZATION` vs `THREEDS`).
- Implementar reglas 3DS faltantes: expiracion, intentos e idempotencia.
- Persistir trazabilidad 3DS diferenciada de tokenization.

14.2. Impacto por componente

Componente/archivo	Cambio esperado	Rango LOC	Riesgo
`NotificationTokenizationController`	Ruteo por `useCase`, validaciones minimas 3DS, normalizacion de respuesta	20-50	Medio
`INotificaionService` + `NotificationService`	Nuevos metodos/ramas para flujo 3DS, reglas de expiracion, intentos, idempotencia, estado funcional 3DS	120-220	Alto
`SendPassCodeTokenization` (y/o nuevo DTO 3DS)	Campos de correlacion 3DS (`threeDSServerTransID`, `dsTransID`/`transactionId`, TTL, max intentos)	20-60	Medio
`INotificationRepository` + `NotificationRepository`	Nuevas operaciones de persistencia/consulta para intentos y traza 3DS (evitar mezclar solo con `saveTokenizationNotification`)	80-160	Alto

Componente/archivo	Cambio esperado	Rango LOC	Riesgo
Persistencia DB (SP/tablas)	SP nueva o extendida para traza 3DS + estado de intentos/expiracion + idempotencia	60-140	Alto
Configuracion (`notificationCategories`)	Nueva categoria/flags 3DS y feature flag por tenant	10-30	Bajo
Pruebas (unitarias/integracion)	Casos por flujo `TOKENIZATION` y `THREEDS` + regresion de sendPassCode actual	120-240	Alto

14.3. Totales estimados

Metrica	Estimacion
Archivos Java impactados	6-10
Archivos de configuracion impactados	1-3
Objetos de DB impactados (SP/tablas/vistas segun implementacion)	2-5
Lineas de codigo (Java + config + scripts DB, sin tests)	310-660 LOC
Lineas de pruebas (recomendado)	120-240 LOC
Esfuerzo relativo	Medio-Alto

14.4. Escenarios de esfuerzo

Escenario	Descripcion	Impacto aproximado
Minimo viable	Solo enrutamiento 3DS + envio OTP + traza basica, con controles parciales	250-380 LOC

Escenario	Descripcion	Impacto aproximado
Recomendado	Incluye expiracion, intentos, idempotencia, trazabilidad completa y pruebas separadas	430-780 LOC

La mayor incertidumbre de esfuerzo esta en capa de persistencia y definicion funcional final de estados 3DS. Si se exige auditoria fuerte por regulacion, el impacto se desplaza al rango alto.